Book 6

Advanced Playbook

Paste-ready production moves this app can finish.

The five guides tell you what Genius is, what to try, what to type, where it stops, and what every action does. This book is only the prompting. Each example is a technique from the local Music and Audio databases, cut down to the actions in the current tool list. If a famous trick needs a plugin load, a follow action, a clip fade, a freeze, or an export, it is not here.

In this book

- A prompt shape that reads, acts, and reads back
- Harmony the chord tools can spell
- Euclidean rhythm, ghosts, polyrhythm, and real grooves
- Drop-first arrangement, then removal
- Drift, Utility, returns, and parallel compression
- Low end checked with a real recording
- Motivic copies that leave the source alone
- A session grid without follow actions
- Samples, file edits, and old .als files
- One D Dorian session, five checkpoints

Techniques from the Production and DJing, Music Theory, Sound Design, and Ableton Training databases on this machine. Action names checked against the app at build time. These prompts were not each run inside a live set for this edition. Paste one, then believe the read-back.

What was left out on purpose

These are the requests that sound like the next chapter of a production book and do not survive contact with this app. Use the substitute, or leave the job to Live.

Request	What happens	Paste-ready substitute
Load Serum, Massive, or a factory preset	Native insert only. The bridge has no browser. A preset load needs the separate Remote Script, and this book does not depend on it.	Insert Drift, Analog, or Wavetable. Read the parameter list. Set only names that exist.
Sidechain the bass to the kick inside the Compressor	The app can insert Compressor and set threshold, ratio, attack, and release. It cannot point the sidechain input at the kick.	Read the kick notes. Draw volume automation on the bass clip that dips where those kicks land.
Set follow actions, or fade an audio clip in Live	Those clip properties were missing on the probed Live version. The call looks fine and changes nothing.	Build the next section as another scene. Fade a file with the audio op, then load the new file as a new clip.
Freeze, flatten, consolidate, or export the song	Not in Live's scripting surface.	Record a timed capture for analysis. It is a measurement, not a master.
Make the macro say Magnitude	The dry/wet binding works. The rename does not.	Map the blend. Rename the macro yourself.
What does this sound like?	The bridge carries state, not audio, until you record.	Record, analyze, and ask about the numbers.

Key names

When a prompt says the key is C, F#, or Bb, that is a requirement of the voicing and walking-bass tools, which reject "A minor" and "D dorian" as a key string. Mode lives in the chord symbols and in the Set scale, which does accept "D Dorian". Slash chords such as V/V are not tokens. Spell them: D7, not V/V.

How to prompt at this level

Book 2 is what to try. Book 3 is a catalog. This book is the next step: production moves taken from the local Music and Audio databases, rewritten as prompts that the current tool list can finish. A prompt belongs here only when every musical result is a real action.

The shape of a prompt that survives

Advanced work fails when the model narrates a change it never made, or guesses a parameter range. The prompts below all share one shape: read the live state, do one musical job, then read it back.

> *Work in three beats and stop between them if a beat fails. One: read the tracks, devices, or clip you are about to touch and quote what you found. Two: make the change with the real tools, and if a parameter name or chord token is rejected, stop and say which one. Three: read the same thing back and list what actually changed. Do not describe a change you did not read back.*

Does: live_read_set, live_read_track, live_read_clip, live_session_diff

Check: The reply quotes a before and an after. A summary with no read-back is not done.

Snapshot before a chain

The longer the chain, the more a middle step can succeed in the reply and fail in the Set. Snapshot and diff are the receipt.

> *Snapshot this session. Then do only the next prompt I paste. When you finish, diff the snapshot and list every track, device, clip, and parameter that actually changed, in plain language. If the diff is empty, say the prompt did not land.*

Does: live_session_snapshot, live_session_diff

Check: You get a list of real paths, not a story.

Harmony you can hear in the clip

The theory database treats modal interchange as borrowing from the parallel mode, and the electronic-harmony notes treat a secondary dominant and a tritone substitute as color on the way home. The chord tools write those as real notes. They do not understand slash chords such as V/V, and live_generate_chords only accepts a key that is a note name such as C, F#, or Bb. Spell the color as a chord symbol.

Borrowed color in C

A major loop that borrows Fm, Ab, and Bb is the parallel-minor color the theory notes describe: minor iv, flat VI, flat VII. Naming the chords avoids a numeral the parser would flatten twice.

> *On a new MIDI track named Chords, write an 8-bar clip in slot 1, key of C. One bar each, seventh voicing: Cmaj7, Fm7, Abmaj7, Bbmaj7, and repeat. Read the notes back and list the chord roots you actually wrote. If any token is rejected, stop. Do not substitute a different chord.*

Does: live_create_track, live_create_chord_progression, live_read_clip

Check: The clip contains C, F, Ab, and Bb as roots, each for one bar, then the same four again.

Drop-2, shell, and rootless of the same changes

live_generate_chords voice-leads a progression in five styles. Rootless drops the root and adds the 9th, so it only works on seventh chords. Shell keeps the root, the 3rd, and the 7th. Drop-2 drops the second voice from the top by an octave.

> *Create three clips on the Chords track, slots 1, 2, and 3, all in the key of C, two bars per chord: Dm7 G7 Cmaj7. Slot 1 drop-2. Slot 2 shell. Slot 3 rootless. Use a strum of 0.02 beats on the drop-2 only. Read each clip and tell me the lowest and highest note in each, so I can hear that they are the same changes in three registers. Do not overwrite a slot you have already filled.*

Does: live_generate_chords, live_read_clip

Check: Three clips, same chord roots, different pitch spreads. Rootless has no root in the bass.

Secondary dominant, spelled out

V/V in C is D7. The tools have no slash-chord token. D7 G7 Cmaj7 is the same idea and both chord writers accept it.

> *In slot 4 of the Chords track, key of C, write D7, G7, Cmaj7, Cmaj7, one bar each, drop-2 voicing. Name the clip Secondary dominant. Read it back. If you cannot spell D7, stop rather than writing D minor.*

Does: live_generate_chords, live_update_clip, live_read_clip

Check: The first chord is a major triad plus a minor 7th on D, not Dm7.

Tritone substitute for the dominant

The electronic-harmony notes describe replacing G7 with the dominant a tritone away, Db7. Flat-II7 is in the roman table. Db7 is the symbol that cannot be misread.

> *In slot 5, key of C, write Dm7, Db7, Cmaj7, one bar each, shell voicing. Name the clip Tritone sub. Then tell me the interval between the root of bar 2 and the root of a normal G7, which should be a tritone. Read the notes before you answer.*

Does: live_generate_chords, live_read_clip

Check: Bar 2 root is Db. The shell has a root, a 3rd, and a 7th.

Walking bass under those changes

The walking-bass generator anchors the root, steps diatonically, and can add ghost notes. Its key argument is a note name, same as the voicing tool. Give it the chord symbols, not a mode name.

> *On a new MIDI track named Walk, slot 1, write a walking bass under Dm7-G7-Cmaj7-Cmaj7 in the key of C, one bar per chord, low octave, ghosts on. Read the clip and confirm the downbeat of each bar is the chord root (D, G, C, C). If a downbeat is wrong, say so and do not quietly rewrite it.*

Does: live_create_track, live_generate_walking_bass, live_read_clip

Check: Quarter-note motion, roots on the downbeats, a few shorter ghost notes.

Hear the mode, then borrow inside it

Key detection is Krumhansl-Schmuckler on the notes that are already in the Set. Borrowing only means something after that result exists.

> *Detect the key of the clip in Chords slot 1. Tell me the key and how confident the result is. If it is not clearly C major, stop and show me the pitch classes. If it is C major, write one more clip in slot 6 that keeps Cmaj7 and replaces one bar with a borrowed Fm7. Name that clip Borrowed iv. Read both clips and point at the bar that changed.*

Does: live_detect_key, live_create_chord_progression, live_read_clip

Check: You see a confidence number before any new notes, and only one bar differs.

Rhythm systems

Euclidean rhythm here is Bjorklund: a fixed number of hits spread evenly across a fixed number of steps, written as MIDI, not described. Drum grids carry velocity in the character (x loud, o medium, g ghost, a caret for an accent). A drum-pattern row with a different number of steps than the others is a polyrhythm, because each row divides the same bar. Groove can only be attached when the Groove Pool already has one.

Three Euclidean voices, rotated apart

Even distribution is the point of the algorithm. Rotating one voice against another is how the same density stops lining up on beat 1.

> *Read the drum rack on the track named Drums. If there is no drum rack, create a MIDI track named Drums and insert a Drum Rack, then read the pads. Using only pads that have a sample, write three one-bar clips in slots 1, 2, and 3. Slot 1: kick, 4 pulses in 16 steps. Slot 2: closed hat, 5 pulses in 16, rotated by 2 steps. Slot 3: snare or clap, 3 pulses in 8 steps, two bars long. If a pad is empty, skip that voice and say so. Do not invent a General MIDI note that the rack does not have.*

Does: live_read_drum_pads, live_create_track, live_create_device, live_generate_euclidean

Check: Hits are even, not a copied 4/4 grid. Empty pads were not used.

Ghost notes you can see in the grid

The grid writer maps g to a quiet hit and a caret to an accent. That is the velocity contour a hat line needs, and it is deterministic.

> *On the Drums track, slot 4, write this one-bar grid at 16th-note steps with swing 0.35. Use the pad notes you already read. If the rack uses different notes, map Kick, Snare, and Hat to those pads and show me the map.*
>
> `Kick: x...x...x...x...`
>
> `Snare: ....x..g....x..g`
>
> `Hat: x.o.x.g.x.^x.o.x`
>
> *Read the clip and tell me the quietest and loudest velocities that landed.*

Does: live_write_drum_grid, live_read_clip

Check: Ghosts are near velocity 45, accents near 125, and the off-16ths are swung.

A 5 against a 4 in one clip

create_drum_pattern divides the bar by the length of each row. A 16-step kick and a 5-step hat in the same 4-beat clip is the polyrhythm.

> *On Drums slot 5, write a 4-beat pattern. Kick on sixteenths as x...x...x...x... and a hat voice as x.x.x (five steps across the same four beats). Use the pad notes from the rack. Read the hat start times back and confirm they are not sitting on a sixteenth grid.*

Does: live_create_drum_pattern, live_read_clip

Check: Five hat onsets inside a bar of four kicks, evenly spaced against each other.

Swing, then a groove that is actually loaded

Global swing and the Groove Pool are different systems. Swing affects quantized timing. A groove attaches only if live_groove_pool lists one. Committing bakes the current groove into the note starts.

> *Set global swing to 0.52. Then list the Groove Pool. If it is empty, say so and stop on the groove half. Do not invent a groove name. If a groove is listed, attach the first one to Drums slot 4 and set the global groove amount to 0.6. Read the clip timing, then commit that groove into the notes at 16th-note strength 0.7, and read the timing again so I can see the starts moved.*

Does: live_set_swing, live_groove_pool, live_set_groove_amount, live_commit_groove, live_read_clip

Check: Either a real groove name from the pool, or an honest empty pool. Timing numbers change after the commit.

Humanize a copy, keep the machine original

Transform replaces the notes in the clip it is aimed at. The original survives only if you duplicate first.

> *Duplicate Drums slot 1 into slot 6. Humanize only slot 6, a small amount. Leave slot 1 untouched. Read both clips and show me two note start times that differ and two that would have been identical if you had edited the original.*

Does: live_duplicate, live_transform_clip, live_read_clip

Check: Slot 1 is still on the grid. Slot 6 has moved starts and velocities.

Grow the loop, then knock a hole in it

Duplicate Loop doubles length and repeats the notes. Removing a pitch range in a time window is how a later bar becomes a fill or a gap without redrawing the whole beat.

> *On a copy of the kick clip, duplicate the loop once so it is two bars, then again so it is four. In bars 3 and 4 only, remove the snare notes if they are in that clip, or remove every hit on the snare pitch. Leave bars 1 and 2 intact. Read the clip and confirm bar 1 still matches the source.*

Does: live_duplicate, live_duplicate_loop, live_remove_notes, live_read_clip

Check: The clip is four bars. The first bar still matches the one-bar source.

Arrangement as a form

The EDM production notes describe a form you can build by hand in Session view and then lay onto the timeline: intro, breakdown, buildup, drop, outro. They also say to finish the drop first and make the earlier sections by taking elements away. Filter rises and snare acceleration are the buildup. Nothing here exports audio. Placement is locators, duplicated clips, and note removal.

Name the form before you fill it

Locators are the map. Scenes are the material. Doing the names first keeps a later prompt from guessing bar numbers.

> *Set the tempo to 126 and the time signature to 4/4. Create locators at bar 1 named Intro, bar 17 named Breakdown, bar 33 named Build, bar 41 named Drop, and bar 73 named Outro. Create five scenes with those same names, in that order. Read the locators and the scene names back. Do not add instruments in this step.*

Does: live_update_set, live_create_scene, live_read_set

Check: Five locators and five scenes, same names, no new devices.

Write the drop, then remove your way backward

The remix notes say the drop is the full idea, the breakdown is that idea with the kick gone and a filter closing, and the intro is thinner again. Removal is a real edit. A filter move waits until the parameter range has been read.

> *Using the drum and chord clips you already have, duplicate the busiest drum clip and the main chord clip into the scene named Drop. Duplicate those same clips into the scene named Breakdown, then in the Breakdown copies remove every kick note and every bass note below C3. Duplicate a hats-only version into the scene named Intro by removing kick, snare, and any chord note in that intro copy. Read all three scenes and tell me the note counts. Do not delete the Drop.*

Does: live_read_scene, live_duplicate, live_remove_notes, live_read_clip

Check: Drop has the most notes. Breakdown has no kick. Intro is hats, or whatever voice you did not remove.

Lay the scenes on the timeline

duplicate_to_arrangement takes a session clip and a beat position. At 4/4, bar 41 is beat 160 because bar 1 starts at beat 0. Ask it to compute the beats from the locators it just read, not from memory.

> *Read the locators. For each of Intro, Breakdown, Build, and Drop, duplicate that scene's clips onto the arrangement at the locator's beat position. The Build can reuse the Breakdown clips for now. Read the arrangement clips on each track and confirm the start times match the locators. If a scene has no clips, skip it and say so.*

Does: live_read_set, live_duplicate_to_arrangement, live_read_arrangement_clips

Check: Arrangement clips start at the locator times. Empty scenes are reported, not invented.

A filter rise only after the range is known

Shaped automation writes a sine, triangle, saw, square, or sample-and-hold into a clip envelope. A filter frequency is often in Hertz, not 0 to 1. The default 0 to 1 range is wrong for that parameter. The prompt has to read the range first. This is the buildup the production notes describe as a filter opening into the drop.

> *On the chord clip that sits in the Build, insert an Auto Filter if the track does not already have one. Read the frequency parameter's minimum and maximum. Draw one saw cycle across that clip from the low end of the real range to about 70 percent of the real range. If the frequency parameter cannot be automated, stop and name the parameters that can. Read the envelope back.*

Does: live_create_device, live_read_param_range, live_add_shaped_automation, live_get_clip_automation

Check: The envelope exists, and its values sit inside the range you were shown.

Loop the drop against the build

The arrangement loop brace is how you audition a join without listening through the intro every time.

> *Read the Build and Drop locator times. Set the arrangement loop to start 4 bars before the Drop and to end 8 bars after the Drop starts. Turn the loop on. Jump the playhead to the loop start and do not start playback unless I ask. Tell me the loop in bars and in beats.*

Does: live_read_set, live_set_loop_region, live_jump_to_time

Check: The loop brace wraps the join. Transport stays stopped.

Native sound, not a plugin

The sound-design notes teach subtractive path on Drift or Analog: one saw, a low-pass, an amp envelope, then a filter envelope. The drum-and-bass notes describe a sub as a sine with no unison, a short attack, and a mono utility, and a Reese as two detuned saws into a low low-pass. Live Actions inserts those native devices by name. It does not load Serum, Massive, or a factory preset. Parameter names that do not exist are skipped, and the prompt must say so.

A sub that stays one note wide

Two subs in the same octave is the low-end problem the mixing notes keep returning to. One sine, mono, short attack, is the version this app can build.

> *Create a MIDI track named Sub. Insert Drift. Read its parameters. Set one oscillator as close to a sine as the parameter list allows, turn unison or a second oscillator down if those parameters exist, and set the amp attack short and the sustain high if those parameters exist. Insert Utility and set its width or mono parameter if it exists. Write a clip of whole notes on the roots of the chord clip, around MIDI note 38. List every parameter you set and every name that was rejected. Do not claim a Reese, and do not load a plugin.*

Does: live_create_track, live_create_device, live_read_device, live_read_param_range, live_update_device, live_write_notes

Check: One Drift, one Utility, root notes only. Rejected parameter names are listed.

Reese recipe, then the truth about what landed

The bass/reese recipe creates a Drift, best-effort sets detune and filter fields, adds Chorus-Ensemble, and writes an 8-beat bass clip. Several of those parameter steps are optional because Drift's display names do not always match the recipe. The musical result is the clip plus whatever parameters actually stuck.

> *List recipes and apply bass/reese only if it is available. If it is not available, say so and stop. After it runs, read the Drift and the Chorus-Ensemble and tell me the parameter values that are actually on the device. Then read the MIDI clip and tell me the pitches. Do not describe the recipe's intent as if it were the device state.*

Does: live_list_recipes, live_apply_recipe, live_read_device, live_read_clip

Check: A Drift track named Reese, a chorus, a MIDI clip, and a list of real parameter values.

Chord stabs, then an arpeggio of the same notes

The electronic-harmony notes use short chord stabs as rhythm. Note length here is just the duration written into the clip. An Arpeggiator is a native MIDI effect and can sit on a copy so the stab clip stays intact.

> *On the Chords track, write a new clip of Cmaj7, Am7, Dm7, G7, each a dotted eighth long, separated so the bar still breathes. Name it Stabs. Duplicate that clip to the next slot. On the duplicate's track, insert an Arpeggiator if one is not already there, read its rate and gate parameters, and set a 16th-note rate only if that parameter exists. If inserting a second arpeggiator would affect the stab clip too, duplicate the whole track first and put the arpeggiator only on the copy.*

Does: live_create_chord_progression, live_duplicate, live_create_device, live_read_param_range, live_update_device

Check: One clip is short chords. The arpeggiated version is a different clip or a different track.

Lock the Set to the mode you are writing in

The Set scale and the Scale MIDI effect are both real. Together they stop later MIDI from drifting out of D Dorian while you edit by hand.

> *Set the Live Set scale to D Dorian. On the Chords track insert a Scale device if one is not there. Read its parameters and set the base and the mode to Dorian if those parameters exist. Write a one-bar melody clip using only D, F, A, and C. Read it back and confirm none of the notes left that set.*

Does: live_update_set, live_create_device, live_read_param_range, live_write_notes, live_read_clip

Check: The Set scale reads D Dorian. The melody clip uses only those four pitches.

Space on returns, not on every track

A return is a native track type. Reverb and Echo are native devices. Sends are a mixer value. That is the whole send architecture.

> *Create two return tracks named Verb and Echo. Insert Reverb on Verb and Echo on Echo. Read each device and set a medium decay or time only if the parameter exists. On the Chords track, set send A to about 0.25 and send B to about 0.12, and tell me which send letter landed on which return. Read the track back.*

Does: live_create_track, live_create_device, live_read_device, live_update_track, live_read_track

Check: Two returns, two devices, and two send values you can see on the chord track.

Parallel compression is a dry/wet under 100 percent

The compression notes call a wet mix below 100 percent parallel, or New York, compression. The racks/parallel-comp recipe loads a native Compressor and sets a heavy threshold with the mix around half.

> *On the Drums track, list recipes and apply racks/parallel-comp only if it is available, with the wet mix at 0.45. Then read the Compressor. Tell me the threshold, ratio, and dry/wet that are actually stored. If dry/wet is still 100 percent, say the parallel move did not land. Do not insert a second compressor to hide that.*

Does: live_list_recipes, live_apply_recipe, live_read_device

Check: One compressor, dry/wet near 45 percent, or an honest report that the mix parameter did not move.

Map the rack blend, then rename it yourself

Mapping Dry/Wet knobs inside a rack to Macro 1 works. Renaming that macro to Magnitude does not. The prompt should expect the first and not the second.

> *Find the first Audio Effect Rack in the Set. Map its chain blend parameters to Macro 1. Read the macros back. Tell me the macro's current name and value. Leave the name alone. I will rename it in Live.*

Does: live_read_set, live_map_rack_magnitude, live_read_macros

Check: Macro 1 moves the dry/wet controls. Its name is unchanged.

Low end and a number, not an opinion

Genius does not hear the song. It can solo a track, record the output through the loopback device, and read LUFS, true peak, and a 10-band spectrum. The mixing notes say to leave one sub in the bottom and to cut the kick's fundamental out of everything else that lives down there. The measurement is what makes that instruction checkable. A capture is not an export.

One occupant under 80 Hz

Structural audit flags two tracks sharing the low end. It is not a listening test. Use it to decide who gets the high-pass, then confirm with a recording.

> *Run a structural mix audit and quote every low-end warning. Then, on every track except the one named Sub and the one that holds the kick, insert an EQ Eight if it does not already have one. Read the high-pass or low-cut parameter. If it exists, set it near 90 Hz using the parameter's own units. Read each EQ back. Do not high-pass the kick or the Sub.*

Does: live_analyze_mix, live_create_device, live_read_param_range, live_update_device, live_read_device

Check: Kick and Sub are untouched. Other tracks show a low cut, or you are told the parameter was missing.

Find the kick's band, then cut it elsewhere

The 50 mixing tips say to subtract the kick's fundamental from the other low elements. The app can only know that frequency by recording the kick. Solo is exclusive, and the capture takes as long as the audio.

> *Exclusively solo the kick track. Record about 8 seconds and analyze it. Tell me the strongest spectrum band below 100 Hz. Unsolo. On the Sub track's EQ Eight, read the parameter list and cut that band with a narrow dip only if a frequency and gain parameter exist for a band. Then record 8 seconds of kick plus sub, analyze again, and tell me whether energy in that band went down. If the loopback device is not available, stop before the first record and say so.*

Does: live_batch_tracks, live_record_and_analyze, live_read_param_range, live_update_device

Check: Two measurements, with a band named in Hz. No fake listening.

Bass glue, then the master chain

bass-glue is Channel EQ, Saturator, and Glue Compressor on the bass track. master-bus is Glue, EQ Eight, and Limiter on the master. Both are native chains. Apply them only when the recipe list says they are available.

> *List recipes. Apply mixing/bass-glue to the Sub track if it is available. Apply mixing/master-bus if it is available. Read both chains in order and write down the devices you find, in signal order. If a recipe is unavailable, skip it and say which requirement is missing. Do not rebuild it from memory with different devices.*

Does: live_list_recipes, live_apply_recipe, live_read_track

Check: Device order matches the recipe, or the recipe is reported unavailable.

A loudness target with a cap on attempts

Each capture is real time. Four attempts is the limit so a missed parameter cannot loop. The number after each attempt is the result, not the model's impression.

> *Play the arrangement loop if it is set, otherwise do not hunt for a better section. Record about 10 seconds and analyze. If integrated LUFS is already between -16 and -12, stop and report the number and the true peak. If it is hotter than -12, read the Limiter ceiling or gain, lower it by a small step inside its real range, and measure again. Do this at most 4 times. Show the LUFS after each attempt. If there is no limiter, say so and do not add a plugin.*

Does: live_read_device, live_read_param_range, live_update_device, live_record_and_analyze

Check: A short table of attempt, LUFS, and true peak. Playback time was actually spent.

Where the audit and the recording disagree

The interesting fault is often the gap between a Set that looks fine and a capture that is bright, or the reverse.

> *Run the structural mix audit. Record 15 seconds and analyze. Then tell me one warning the audit raised that the spectrum does not support, and one spectral fact the audit could not have known. Do not change any device in this step.*

Does: live_analyze_mix, live_record_and_analyze

Check: Two sources, and a sentence on where they differ. The Set is unchanged.

Duck the sub on the kick, without a sidechain input

The compressor's sidechain chooser is not a tool. Clip volume automation is. A dip at each kick start is the pump the production notes mean, locked to the notes you can read.

> *Read the kick clip and list each kick start time. On the Sub clip in the same scene, write a volume envelope that rests near full level and dips at every kick start, recovering before the next kick. Use the kick times you just read, not an even sine. Read the envelope back and tell me whether the number of dips matches the number of kicks. If clip volume cannot take an envelope, stop and say so. Do not turn on the compressor sidechain and claim the kick is routed.*

Does: live_read_clip, live_set_clip_automation, live_get_clip_automation

Check: One dip per kick, at the kick's start time, or an honest stop.

A 15 ms double, panned

Track delay is a real mixer value in milliseconds. A duplicated music track, panned the other way, delayed a little, is a width move that does not require a widener plugin. Keep it off the kick and the sub.

> *Duplicate the Chords track. Name the copy Chords Wide. Pan the original 20 percent left and the copy 20 percent right. Set the copy's track delay to 15 ms. Read both mixer strips back. Do not do this to Drums or Sub.*

Does: live_duplicate, live_update_track, live_read_track

Check: Two chord tracks, opposite pans, one of them delayed by 15 ms.

Develop one idea instead of writing six

Inversion, retrograde, augmentation, diminution, and transposition are operations on the notes in a clip. Each one replaces the clip it targets. The way to get a family of variations is to duplicate, then transform the copy. Selected-note transform is not used here. It only works when notes are already selected in the piano roll, and that path is the least proven one in the tool list.

Six versions, one source

Retrograde reverses time. Inversion reflects pitch around an axis, the median note unless you pass one. Augmentation doubles the durations. Diminution halves them. Transposition is a semitone shift.

> *Pick the melody or stab clip with the most identifiable shape. Duplicate it into the next five empty slots on the same track. Leave the source slot alone. On the five copies, apply in order: invert, retrograde, retrograde invert, augment by 2, and transpose up 7 semitones. Name the clips so the operation is in the name. Read the source and two of the copies and show me one pitch or start time that proves the operation happened.*

Does: live_read_clip, live_duplicate, live_transform_clip, live_update_clip

Check: Six clips. The first still matches the original. The names match the operations.

A sequence by stacking transpositions

A diatonic sequence is the same shape started on the next chord tone. The tool transposes by semitones, so ask for the intervals in semitones and keep each copy in its own slot where you can hear them.

> *Duplicate the source phrase into three new slots. Transpose the copies by +2, +5, and +7 semitones. Do not transpose the source. Read all four and list the first note of each.*

Does: live_duplicate, live_transform_clip, live_read_clip

Check: Four first-notes, each a stated interval above the source.

Call, then the answer, in one clip

The transform tool cannot append. It replaces. The way to put a phrase and its retrograde in one clip is to transform a copy, read both, and write a new clip whose second half starts later.

> *Read the source phrase. Duplicate it to a scratch slot and retrograde only the scratch slot. Read both. Create a new clip that contains the original notes in the first half and the retrograde notes in the second half, with the answer starting at the end of the question. Delete the scratch slot when the new clip reads back correctly. If the combined length will not fit in the clip, stop and tell me the two lengths.*

Does: live_read_clip, live_duplicate, live_transform_clip, live_write_notes, live_delete

Check: One clip, question then answer. The scratch slot is gone.

Make the session playable

Launch mode, launch quantization, and legato are clip properties that stick. Crossfader assignment is a track property. Follow actions are not in this book: the clip properties they write were absent on the Live version this app was probed against, so a prompt that sets them looks successful and changes nothing. Scenes plus launch mode are the substitute.

Four energy states, one grid

A performance grid is scenes as versions of the track, not as a copy of the arrangement. The launchpad recipe builds empty tracks and empty scenes when it is available. The same result is four named tracks and eight named scenes if the recipe is gated.

> *List recipes. If live_performance/launchpad-rig is available, apply it at 126 BPM with 8 scenes, then rename the scenes Intro, Groove A, Groove B, Break, Build, Drop, Drop Alt, Outro. If it is not available, create four MIDI tracks named Drums, Bass, Music, FX and eight scenes with those names. Set global clip quantization to one bar. Read the scene list back.*

Does: live_list_recipes, live_apply_recipe, live_create_track, live_create_scene, live_update_set, live_read_set

Check: Eight named scenes and four named tracks. Quantization is one bar.

Toggle the grooves, legato the pads

Toggle with one-bar quantization is how a clip stays on until you press it again. Legato is how a pad clip picks up the transport position instead of restarting.

> *On the Music track, set every clip that exists to legato with one-bar launch quantization. On the Drums track, set every clip to toggle with one-bar quantization. Read one clip from each track and quote the launch mode, the quantization, and the legato flag. If a clip has no such property in the read-back, say so.*

Does: live_read_track, live_set_clip_launch_mode, live_read_clip

Check: Drums clips toggle. Music clips are legato. Both quantize to a bar.

Crossfader as two decks of your own track

Assigning tracks to side A or B is a mixer write. The crossfader position is a master write. Role comes from the track name and the devices you read, not from a guess about the sound.

> *Read the tracks. Put any track whose name or devices say drums, kick, or bass on crossfader A. Put chords, music, and pads on crossfader B. Leave returns unassigned. Set the crossfader to the center. Read the assignments back.*

Does: live_read_set, live_update_track, live_set_crossfader, live_read_track

Check: Drums and bass on A, music on B, crossfader at center.

Capture a combination you just launched

create_scene can copy whatever is currently playing in Session view into a new scene. That is the rescue for a clip combination you will not remember.

> *I am going to launch a combination. When I say so in the next message, create a scene named Captured that copies the currently playing session clips. Do nothing until I say the combination is playing. If nothing is playing when I ask, say so and do not create an empty scene.*

Does: live_create_scene, live_read_scene

Check: A new scene whose clips match what was playing, or a clear refusal.

Files, samples, and old sets

These run even when you are not trying to be musical. Sample search reads the Samples folder on disk. Audio ops write a new file and never overwrite the source. An .als can be summarized, linted, diffed, and mined for MIDI with Live closed. Writing that MIDI into the open Set is a separate, explicit step.

A kick from disk, not from the browser

find_samples searches the configured Samples folder. create_clip with a file path puts that file on an audio track. The Live browser is not required for this path.

> *Search my samples for a kick. Show me up to five paths. Stop and let me pick one if there is more than one. When there is a single obvious kick, or after I name one, create an audio track named Kick Audio and put that file in slot 1 as a one-shot clip. Read the clip back and quote the sample name and the clip length.*

Does: live_find_samples, live_create_track, live_create_clip, live_read_clip

Check: An audio clip whose sample path is one of the paths that was listed.

Reverse it, pitch it, keep the original

reverse and pitch-time always write a new file. The original sample stays where it was. Loading the new file is a second clip.

> *Take the kick audio clip's file. Write a reversed copy next to it, then a second copy pitched down 12 semitones at the same tempo. Put each new file in its own slot on Kick Audio. Read the three slots and quote the three file names. Do not overwrite slot 1.*

Does: live_read_clip, live_audio_op, live_create_clip

Check: Three slots, three files. Slot 1 still points at the original.

A texture from noise, then a crush

The file tools can generate noise and can bitcrush. That is a way to make a bed without a plugin and without a sample pack.

> *Generate 8 seconds of brown noise at a low amplitude. Bitcrush a copy of that file to 8 bits with the mix around half. Put the crushed file on a new audio track named Texture, slot 1. Read the clip and quote the path. Leave the uncrushed noise file on disk and out of the Set.*

Does: live_audio_op, live_create_track, live_create_clip, live_read_clip

Check: One audio clip, crushed file. The clean noise file is not in the Set.

Normalize a capture you already made

normalize targets integrated LUFS or a peak ceiling and writes a new file. compare_audio then tells you the delta against the file you recorded.

> *Find the newest recording you captured in this session. If there is none, stop. Normalize a copy to -14 LUFS with a true-peak ceiling of -1.5. Compare the normalized file to the original and report the LUFS delta and the true-peak delta. Do not replace anything inside the Live Set.*

Does: live_audio_op, live_compare_audio

Check: Two files on disk, and a numeric before/after. The Set is untouched.

Which old projects are worth opening

als_find_unfinished lints a saved set for empty clips, a clipping master, a missing limiter, and soloed tracks, without launching Live. Point it at a folder only after you name the folder. Do not scan the whole disk.

> *I will give you one folder of .als files. For each set, summarize tempo, track count, and the unfinished-lint warnings. Rank them by how much real material they already contain. Do not open them in Live. Do not modify the files.*

Does: als_summary, als_find_unfinished, als_list_tracks

Check: A ranked list with warnings quoted from the files. Live can be closed.

Lift a MIDI clip out of an old set

als_extract_midi returns note events from a saved file. Writing them into the open Set is live_write_notes, and it should wait until you have seen the note count.

> *Extract the MIDI from the clip I name in the .als file I name. Tell me the note count, the length in beats, and the pitch range. Do not write it into the open Set until I say yes. When I say yes, write it into the slot I name, replacing that slot only.*

Does: als_extract_midi, live_write_notes, live_read_clip

Check: You see the note count before anything in the open Set changes.

One session, in order

This is the prompts above as a single build. Each block is a checkpoint. Paste them one at a time. The musical target is a D Dorian loop at 126, with a drop written first, a breakdown made by removal, a native sub, and one real loudness measurement.

Checkpoint 1. The frame

Tempo, scale, names. No sound yet.

> *Snapshot the session. Set tempo to 126 and the scale to D Dorian. Create MIDI tracks named Drums, Sub, and Chords, each a different color. Create scenes named Drop, Breakdown, and Intro. Create locators at bar 1 Intro, bar 9 Breakdown, and bar 17 Drop. Read the Set back and diff the snapshot in one short list.*

Does: live_session_snapshot, live_update_set, live_create_track, live_create_scene, live_session_diff

Check: Three tracks, three scenes, three locators, scale D Dorian, a diff that matches.

Checkpoint 2. Drums that are not a preset loop

Even kicks, a swung hat with ghosts, and a hat that argues with the kick.

> *Insert a Drum Rack on Drums and read the pads. In the Drop scene, write a 4-pulse 16-step kick on a loaded kick pad, and a hat pattern with accents and ghosts using the grid characters, swing 0.4. In another slot in that scene, write a 5-step hat against the 4-beat bar on a loaded hat pad. Skip any voice whose pad is empty. Read the notes back.*

Does: live_create_device, live_read_drum_pads, live_generate_euclidean, live_write_drum_grid, live_create_drum_pattern, live_read_clip

Check: Only loaded pads. Kick on the sixteenths. One hat line swung, one hat line in five.

Checkpoint 3. Dorian changes and a monotone sub

D Dorian's characteristic color against a normal minor is the major IV. In D that is G major. The loop is Dm7, Gmaj7, Cmaj7, Fmaj7. The sub plays the roots and nothing else.

> *On Chords, in the Drop scene, write Dm7, Gmaj7, Cmaj7, Fmaj7, one bar each, drop-2, key named as a note only where the tool requires a note name. Read the roots back: D, G, C, F. On Sub, insert Drift and Utility. Read parameters before you set them. Aim for a sine, no unison, short attack, and mono width, and list any name the device rejected. Write whole-note roots at MIDI 38, 43, 36, and 41, one bar each, matching those chords. Read the sub clip.*

Does: live_generate_chords, live_create_device, live_read_param_range, live_update_device, live_write_notes, live_read_clip

Check: Chord roots D G C F. Sub is the same roots, one note at a time, and rejected parameter names are listed.

Checkpoint 4. Breakdown by deletion, then the timeline

Same material, fewer notes, then the two scenes placed on the arrangement.

> *Duplicate the Drop clips into the Breakdown scene. In the Breakdown copies, remove every kick and every sub note. Read the note counts and confirm the Drop still has its kick. Duplicate the Drop clips onto the arrangement at the Drop locator, and the Breakdown clips at the Breakdown locator. Read arrangement start times and confirm they match the locators.*

Does: live_duplicate, live_remove_notes, live_read_clip, live_duplicate_to_arrangement, live_read_arrangement_clips

Check: Breakdown has no kick and no sub notes. Arrangement starts line up with the locators. Drop is intact.

Checkpoint 5. One measurement, then stop

Glue and a limiter if the recipes are available, then a single capture. No five-minute mastering session.

> *If mixing/master-bus is available, apply it. Record about 12 seconds of the arrangement and analyze. Report integrated LUFS, true peak, and the strongest band below 120 Hz. Do not change a parameter after the measurement. Diff against the snapshot from checkpoint 1 only if you still have it, and keep the report to what changed.*

Does: live_list_recipes, live_apply_recipe, live_record_and_analyze, live_session_diff

Check: One LUFS, one true peak, one low band. No second attempt unless you paste a new prompt.

After you paste one

Read the check line before you listen. If the read-back does not match it, the prompt is finished and the result is wrong, which is a better place to be than a confident summary. Undo is Live's own undo, and one prompt can be several undo steps. Snapshot first when the Set is one you care about.